Producer-Consumer

ducer-Consumer · Bounded queue between encoder and uploader · Build with Wa

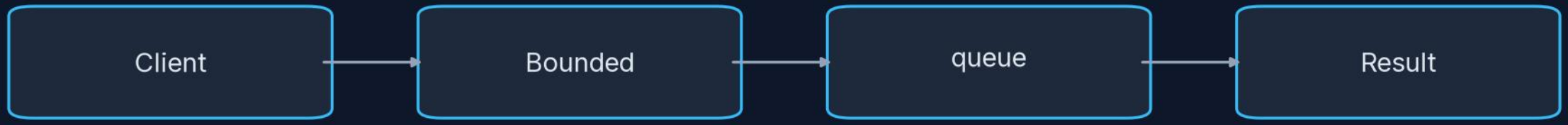
Today's Hook

- Problem: Bounded queue between encoder and uploader
- Pattern: Producer-Consumer — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Bounded queue between encoder and uploader
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Producer-Consumer: Bounded queue between encoder and uploader

Real-World Scenario

- Running example: Bounded queue between encoder and uploader
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.producerconsumer.problem;

import java.util.*;

public final class NaivePipe {
  public static void run(List<String> jobs){ for(String j: jobs) System.out.println("upload "+j); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.producerconsumer.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class BoundedQueue<T> {
  private final ArrayBlockingQueue<T> q;
  BoundedQueue(int cap){ q=new ArrayBlockingQueue<>(cap); }
  void put(T t) throws InterruptedException { q.put(t); }
  T take() throws InterruptedException { return q.take(); }
  int size(){ return q.size(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.producerconsumer.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Producer-Consumer: encoder→uploader queue ===");
    System.out.println("--- naive ---"); NaivePipe.run(List.of("a","b"));
System.out.println("--- producer-consumer ---");
var q=new BoundedQueue<String>(2);
var done=new CountDownLatch(1); List<String> uploaded=new ArrayList<>();
Thread consumer=Thread.ofVirtual().start(() -> { try {
  for(int i=0;i<3;i++) uploaded.add(q.take());
  done.countDown();
} catch(InterruptedException e){ Thread.currentThread().interrupt(); }});
q.put("a"); q.put("b"); q.put("c"); done.await(2, TimeUnit.SECONDS);
System.out.println("uploaded="+uploaded);
  }
}
```

Walkthrough Output

```
=== Producer-Consumer: encoder→uploader queue ===
--- naive ---
upload a
upload b
--- producer-consumer ---
uploaded=[a, b, c]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Producer-Consumer.

Related Patterns

- Contrast Producer-Consumer with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Producer-Consumer → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.